AI CODING AGENT PROJECT REQUIREMENTS

Project Name

AI-Powered Integrated Bid Compliance Verification Platform for GeM Procurement

Problem Statement

SIH 2026 — Problem Statement ID 26100

Goal

Build a production-quality, secure web platform that helps procurement officers automatically analyze tender documents, process bidder submissions, verify compliance against tender requirements, and generate an explainable final compliance report.

The system must NOT simply use an LLM to say "compliant/non-compliant". Every AI decision must be traceable to extracted requirements, submitted documents, extracted data, verification results, and/or human reviewer decisions.

---

1. USER ROLES

Implement strict backend RBAC for:

Admin

- Manage users and roles
- Manage system configuration
- View audit logs
- Manage verification configuration

Bidder

- Create/view own bid submissions
- Upload required documents
- View document processing status
- View compliance results for their own bids
- Never access another bidder's data

Procurement Officer

- Create/manage tenders
- Upload tender documents
- View bidder submissions
- View AI compliance analysis
- Request additional documents

- Approve/reject/finalize bids where authorized

Reviewer

- Review AI-flagged/uncertain cases
- Examine evidence
- Override AI decision with justification
- Add comments
- Finalize assigned reviews

Auditor

- Read-only access to authorized audit information
- View verification history
- View decision trail
- Cannot modify bid/document data

RBAC MUST be enforced in the backend, not only in the frontend.

---

2. MAIN SYSTEM WORKFLOW

Implement this complete workflow:

TENDER CREATION ↓ TENDER DOCUMENT UPLOAD ↓ TENDER REQUIREMENT EXTRACTION ↓ STRUCTURED REQUIREMENT CHECKLIST ↓ BIDDER SUBMISSION ↓ DOCUMENT UPLOAD ↓ FILE VALIDATION + SECURE STORAGE ↓ OCR / DOCUMENT PARSING ↓ STRUCTURED DATA EXTRACTION ↓ REQUIREMENT ↔ DOCUMENT MATCHING ↓ COMPLIANCE ANALYSIS ↓ ENTITY / CROSS-DOCUMENT VERIFICATION ↓ EXTERNAL OR MOCK VERIFICATION WHERE AVAILABLE ↓ RISK / CONFIDENCE ANALYSIS ↓ HUMAN REVIEW IF REQUIRED ↓ FINAL DECISION ↓ COMPLIANCE REPORT ↓ AUDIT TRAIL

---

3. TENDER REQUIREMENT EXTRACTION

When a procurement officer uploads a tender document, the system should extract:

- Eligibility requirements
- Technical requirements
- Financial requirements
- Mandatory documents
- Experience requirements
- Turnover requirements
- Certification requirements
- Registration requirements
- Product/service specifications
- Quantity requirements
- Deadline information
- Validity requirements
- Other conditions

Convert extracted requirements into structured records.

Example:

Requirement: "Bidder must have minimum annual turnover of ₹50 lakh."

Store:

- requirement_type = financial
- requirement_text = original requirement
- parameter = annual_turnover
- operator = >=
- threshold = 5000000
- mandatory = true
- source_document = tender.pdf
- source_page = page number
- extraction_confidence = value

Always retain the original source location wherever possible.

---

4. BIDDER DOCUMENT MANAGEMENT

Allow bidders to upload documents such as:

- GST certificate
- PAN
- Company registration
- MSME certificate
- Experience certificates
- Turnover/financial documents
- Technical certificates
- Product certifications
- Authorization certificates
- Tender-specific documents
- Other supporting evidence

For every file:

- Validate extension
- Validate MIME type
- Validate file size
- Generate unique storage identifier
- Prevent path traversal
- Store metadata
- Record uploader
- Record upload timestamp
- Associate document with bidder and bid
- Maintain processing status

Never trust filenames or client-provided metadata.

---

5. OCR AND DOCUMENT AI

The system must support both:

1. Digital PDFs/documents
2. Scanned documents/images

Processing pipeline:

FILE → TEXT EXTRACTION → OCR IF NECESSARY → DOCUMENT CLASSIFICATION → FIELD EXTRACTION → NORMALIZATION → VALIDATION → DATABASE STORAGE

Extract fields such as:

- Organization name
- Bidder name
- GST number
- PAN
- Registration number
- Certificate number
- Issue date
- Expiry date
- Financial amount
- Address
- Authorized person
- Relevant technical values

Store extracted values together with:

- confidence score
- source document
- page number
- text/evidence location where possible

---

6. COMPLIANCE ENGINE

Create a dedicated compliance engine.

For every tender requirement, determine:

COMPLIANT

Evidence satisfies the requirement.

NON_COMPLIANT

Evidence clearly fails the requirement.

NEEDS_REVIEW

Evidence is missing, ambiguous, contradictory, or AI confidence is insufficient.

Each result must contain:

- Requirement ID
- Requirement text
- Decision
- Reason
- Evidence document
- Evidence page
- Extracted value
- Required value
- Confidence score
- Verification status
- Reviewer decision if applicable

Example:

Requirement: Minimum turnover = ₹50 lakh

Extracted bidder turnover: ₹72 lakh

Result: COMPLIANT

Reason: "Submitted financial document indicates turnover of ₹72 lakh against required minimum of ₹50 lakh."

---

7. CROSS-DOCUMENT VERIFICATION

Implement consistency checks between documents.

Examples:

GST certificate: ABC Technologies Pvt Ltd

PAN: ABC Technologies Pvt Ltd

Registration certificate: ABC Technologies Pvt Ltd

→ Consistent

If one document contains:

ABC Technology Pvt Ltd

→ Flag potential mismatch.

Check:

- Organization names
- Registration numbers
- GST/PAN information
- Certificate numbers
- Dates
- Addresses
- Financial values
- Expiry dates
- Technical specifications

Do not automatically declare fraud from a mismatch. Mark it as a verification/risk issue requiring review where appropriate.

---

8. CERTIFICATE VALIDITY

Where applicable, check:

- Issue date
- Expiry date
- Validity period
- Tender submission date

Example:

Certificate expiry < bid submission date

→ NON_COMPLIANT / NEEDS_REVIEW depending on requirement rules.

The rule engine should be configurable rather than hard-coded wherever practical.

---

9. VERIFICATION SYSTEM

Design a verification abstraction layer.

Possible verification sources:

- Authorized APIs
- Public verification systems
- Organization-provided verification endpoints
- Controlled mock verification for SIH demonstration

Do NOT pretend that mock verification is real government verification.

Every verification record should contain:

- Verification type
- Source
- Request/reference ID where applicable
- Timestamp
- Result
- Status
- Evidence
- Error information

If an external source is unavailable, clearly display:

"Verification unavailable — manual verification required."

---

10. AI AGENT / AI SERVICES

AI can be used for:

- Requirement extraction
- Document classification
- Information extraction
- Semantic matching
- Compliance reasoning
- Anomaly detection
- Summarization

However:

AI must not be the sole source of truth for critical decisions.

Use deterministic rules wherever possible.

Architecture should therefore follow:

AI EXTRACTION

+ RULE ENGINE

+ VERIFICATION

+ HUMAN REVIEW = FINAL DECISION

AI responses must be structured and validated against predefined schemas.

---

11. HUMAN REVIEW

If:

- AI confidence is low
- Documents conflict
- Evidence is missing
- Verification fails
- Requirement interpretation is ambiguous

Then create a review task.

Reviewer should see:

- Tender requirement
- Bidder document
- Extracted information
- Evidence
- AI reasoning/summary
- Verification result
- Recommended decision

Reviewer can:

- Approve
- Reject
- Mark needs clarification
- Request document
- Add comment

Every override must record:

- Reviewer
- Timestamp
- Previous decision
- New decision
- Reason

---

12. DASHBOARDS

Procurement Officer Dashboard

Display:

- Total tenders
- Active tenders
- Total bids
- Compliant bids

- Non-compliant bids
- Pending reviews
- Verification failures
- High-risk/flagged cases

Bidder Dashboard

Display:

- Submitted bids
- Uploaded documents
- Processing status
- Missing documents
- Compliance status
- Review requests

Reviewer Dashboard

Display:

- Assigned reviews
- Pending reviews
- High-priority cases
- AI uncertainty cases
- Completed reviews

Auditor Dashboard

Display:

- Verification history
- Decision history
- Document access history
- Audit events

---

13. REPORT GENERATION

Generate a final Bid Compliance Report.

Report should contain:

1. Tender information
2. Bidder information
3. Requirement checklist
4. Document list
5. Extracted evidence
6. Compliance result
7. Verification result

8. AI confidence
9. Reviewer decisions
10. Exceptions/issues
11. Final recommendation
12. Audit/reference information

The report must be understandable to a human procurement officer.

---

14. DATABASE

Use PostgreSQL.

Design normalized tables/entities for at least:

- users
- roles
- permissions
- tenders
- tender_requirements
- bids
- documents
- document_extractions
- verification_records
- compliance_results
- review_tasks
- review_decisions
- audit_logs
- notifications

Use foreign keys and appropriate indexes.

Never store important relational information only inside JSON blobs when a proper relational structure is appropriate.

---

15. BACKEND

Use:

- Python
- FastAPI
- SQLAlchemy
- PostgreSQL
- Pydantic

Backend should be modular.

Suggested structure:

backend/ ├── app/ │ ├── api/ │ ├── core/ │ ├── models/ │ ├── schemas/ │ ├── services/ │ ├── repositories/ │ ├── ai/ │ ├── verification/ │ ├── compliance/ │ ├── security/ │ └── main.py ├── tests/ ├── migrations/ └── requirements.txt

Keep business logic out of route handlers wherever possible.

---

16. FRONTEND

Build a professional dashboard-based UI.

Required pages:

- Login
- Registration
- Dashboard
- Tender Management
- Tender Details
- Bid Submission
- Document Upload
- Document Processing
- Compliance Results
- Verification Details
- Review Queue
- Reports
- Audit Logs
- User/Role Management

The UI should clearly distinguish:

- AI-generated result
- Rule-based result
- External verification
- Human reviewer decision

---

17. SECURITY

Security is a CORE REQUIREMENT, not an optional feature.

Implement:

- Secure authentication
- Backend RBAC
- Authorization checks on every protected resource
- HTTPS-ready architecture
- Secure password hashing
- Secure secret management

- Environment variables for secrets
- Input validation
- SQL injection protection through ORM/parameterized queries
- CSRF protection where applicable
- Rate limiting where appropriate
- Secure file upload handling
- File size/type validation
- Path traversal protection
- Access-controlled document retrieval
- Encryption in transit
- Encryption at rest where supported
- Audit logging
- Secure error handling
- No secrets in source code
- No sensitive information in logs

A bidder must NEVER be able to access another bidder's documents by changing an ID in an API request.

---

18. AUDITABILITY

Log important events:

- Login/logout
- User creation
- Role changes
- Tender creation
- Document upload
- Document access
- AI processing
- Verification request
- Compliance decision
- Reviewer override
- Report generation
- Administrative changes

Audit logs should be tamper-resistant as far as practical and must contain:

- Actor
- Action
- Resource
- Timestamp
- Result
- Relevant metadata

---

19. API DESIGN

Create clean REST APIs.

Examples:

POST /auth/login

POST /tenders

GET /tenders

GET /tenders/{id}

POST /tenders/{id}/documents

POST /tenders/{id}/extract-requirements

POST /bids

POST /bids/{id}/documents

GET /bids/{id}/compliance

POST /bids/{id}/verify

GET /reviews

POST /reviews/{id}/decision

GET /reports/{bid_id}

GET /audit-logs

All endpoints must enforce authentication and authorization.

---

20. ERROR HANDLING

Implement consistent API error responses.

Handle:

- Invalid files
- Missing documents
- OCR failures
- AI failures

- Verification timeouts
- Database failures
- Unauthorized access
- Expired authentication
- Invalid input
- Duplicate submissions

Do not expose stack traces, database credentials, API keys, or internal implementation details to users.

---

21. TESTING

Create tests for:

Unit Tests

- Requirement extraction
- Rule engine
- Compliance calculations
- Date validation
- Permission checks

Integration Tests

- Authentication
- Tender workflow
- Bid workflow
- Document processing
- Compliance workflow

Security Tests

- Unauthorized API access
- Cross-user document access
- Role escalation
- Invalid file uploads
- Injection attempts

End-to-End Test

Test the complete flow:

Admin creates user → Officer creates tender → Tender requirements extracted → Bidder submits bid → Documents uploaded → OCR/extraction → Compliance analysis → Verification → Reviewer review → Final report generated

---

22. DEMO DATA

Provide realistic synthetic/demo data for SIH presentation.

Do NOT use real confidential procurement or personal data.

Create:

- Sample tender
- Sample bidder
- Sample certificates
- Valid documents
- Invalid/expired document
- Missing document
- Mismatched organization document
- Sample verification responses

This should allow judges to see both successful and failed compliance scenarios.

---

23. IMPORTANT DEVELOPMENT RULES

1. Build the system modularly.
2. Do not put everything in one file.
3. Do not hard-code credentials.
4. Do not hard-code AI responses.
5. Do not claim mock verification is real verification.
6. Do not allow frontend-only authorization.
7. Preserve evidence for every compliance decision.
8. Use deterministic rules wherever deterministic validation is possible.
9. Use AI for extraction/reasoning where appropriate.
10. Always provide human-review fallback.
11. Maintain complete auditability.
12. Design the system so additional verification providers can be added later.
13. Use PostgreSQL as the primary relational database.
14. Keep secrets outside Git.
15. Write clean, maintainable, production-oriented code.

---

24. MVP PRIORITY

Implement in this order:

P0 — Mandatory

- Authentication
- RBAC
- PostgreSQL
- Tender creation
- Tender upload
- Requirement extraction
- Bidder document upload
- OCR/document extraction

- Compliance engine
- Compliance dashboard
- Reviewer workflow
- Audit logs
- Security controls

P1 — Important

- Cross-document verification
- Certificate validity checks
- Verification abstraction layer
- PDF compliance report
- Notifications

P2 — Advanced

- External verification APIs
- Advanced anomaly detection
- Improved AI confidence scoring
- Analytics
- Advanced reporting

FINAL PRODUCT EXPECTATION

The final application should demonstrate:

"Upload a tender → AI extracts requirements → bidder uploads documents → system extracts and validates information → requirements are automatically checked → inconsistencies and missing evidence are detected → available information is verified → uncertain cases go to a human reviewer → final compliance report is generated → every important action is recorded in an audit trail."

The system must be **secure, explainable, auditable, modular, and demo-ready for SIH 2026**.